

Dyn-SSM: Towards the Efficient Long Sequence Learning via Bio-interpretable Dynamics in Spiking State Space Models

(Supplementary Material)

Yan Zhong, Ruoyu Zhao, Chao Wang, Jiaqi He, Qinghai Guo, Jianguo Zhang, Zhichao Lu, Luziwei Leng

I. THE PROOFS

A. The Proof of Assertion III.1

We utilize the Mathematical Induction Theory [1] to prove Assertion III.1. As the presupposition, we set $s_0 = 0$ and $u_0 = 0$, which is reasonable since there's no signal at $t = 0$ with the time step of the spiking signal s , membrane voltage u , refractory momentum R and input current I signal all considered as 1 to L , that is s_t, u_t, R_t and $I_t, t = 1, \dots, L$. On this basis, according to Eq. (6) in the main text ($u_t = \tau u_{t-1} - s_{t-1}U_{th} + I_t, s_t = H_s(u_t - v_{th})$), we have the following derivation.

Firstly, consider $t = 1$, we can obtain:

$$u_1 = \tau u_0 - s_0 U_{th} + I_1 \quad (1)$$

$$\xrightarrow{s_1 = H_s(u_1 - v_{th})} k_1 = I_1, m_1 = v_{th}, \quad (2)$$

which is congruent to Eqs. (7-8) in the main text with $t = 1$. Then, to verify that Eqs. (7-8) in the main text hold for all $t \in [1, L]$, we assume that Eqs. (7-8) in the main text hold for $t = t_1 \in [1, L - 1]$, that is:

$$u_{t_1} = k_{t_1} - m_{t_1} + v_{th}, s_{t_1} = H_s(k_{t_1} - m_{t_1}), \quad (3)$$

$$\text{where } k_{t_1} = \sum_{i=1}^{t_1} \tau^{t_1-i} I_i, \quad \text{and} \quad (4)$$

$$m_{t_1} = U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} s_i + v_{th}, \quad (5)$$

This work is supported in part by the Science and Technology Innovation 2030-Major Project (Brain Science and Brain-Like Intelligence Technology) under Grant 2022ZD0208700.

Yan Zhong and Chao Wang did this work during their internship in ACS Lab, Huawei Technologies Co., Ltd., Shenzhen. Ruoyu Zhao and Jiaqi He did this work during their internship in Department of Computer Science, City University of Hong Kong, Hong Kong

Yan Zhong is with the School of Mathematical Sciences, National Engineering Research Center of Visual Technology, State Key Laboratory of Multimedia Information Processing, School of Computer Science, Peking University, Beijing, China (e-mail: zhongyan@stu.pku.edu.cn).

Ruoyu Zhao and Jiaqi He are with the Department of Electronic Engineering, Tsinghua University, Beijing, China (e-mails: zhao-ry20@mails.tsinghua.edu.cn; he-jq20@tsinghua.org.cn). Chao Wang and Jianguo Zhang are with the Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China (e-mails: wangc2023@mail.sustech.edu.cn; zhangjg@sustech.edu.cn). Qinghai Guo and Luziwei Leng are with the ACS Lab, Huawei Technologies Co., Ltd., Shenzhen, China (e-mails: guoqinghai@huawei.com; lengluziwei@huawei.com). Zhichao Lu is with the Department of Computer Science, City University of Hong Kong, Hong Kong (e-mail: zhichao.lu@cityu.edu.hk).

Yan Zhong and Ruoyu Zhao contribute equally. Corresponding author: Luziwei Leng.

based on this we have:

$$u_{t_1+1} = \tau u_{t_1} - s_{t_1} U_{th} + I_{t_1+1}, \quad (6)$$

$$s_{t_1+1} = H_s(u_{t_1+1} - v_{th}). \quad (7)$$

According to substituting the u_{t_1} in Eq. (3) into Eq. (6), we have:

$$u_{t_1+1} = \tau(k_{t_1} - m_{t_1} + v_{th}) - s_{t_1} U_{th} + I_{t_1+1} \quad (8)$$

$$= \tau \left(\sum_{i=1}^{t_1} \tau^{t_1-i} I_i - \left(U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} s_i + v_{th} \right) \right) \quad (9)$$

$$+ \tau v_{th} - s_{t_1} U_{th} + I_{t_1+1} \quad (10)$$

$$= \tau \sum_{i=1}^{t_1} \tau^{t_1-i} I_i + I_{t_1+1} \quad (11)$$

$$- U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} s_i - s_{t_1} U_{th} \quad (12)$$

$$= \sum_{i=1}^{t_1+1} \tau^{t_1+1-i} I_i \quad (13)$$

$$- \left(U_{th} \sum_{i=1}^{t_1} \tau^{t_1-i} s_i + v_{th} \right) + v_{th} \quad (14)$$

$$= k_{t_1+1} - m_{t_1+1} + v_{th} \quad (15)$$

Therefore, Eqs. (7-8) in the main text hold for $t = t_1 + 1 \in [2, L]$. According to the Mathematical Induction Theory [1], we can conclude that Eqs. (7-8) in the main text hold for all the $t \in [1, L]$, so that Assertion III.1 holds. Q.E.D.

The optimization process of Parallel Max-Min Boundary Compression (PMBC) is shown in Algorithm 1.

B. The Proof of Assertion III.2

We assume that the dimension of the input signal I is L for the refractory LIF neurons with the soft reset. Firstly, according to Eq. (7) and Eq. (8) in the main text, we have:

$$k_1 = I_1, \quad m_1 = v_{th}. \quad (16)$$

Hence we can definitely obtain $s_1 = H_s(k_1 - m_1)$ in the first iteration of PMBC. Then, without loss of generality, for the $(a + 1)$ -th iteration of PMBC, we assume that the first a

spiking signals (i.e. $s_1, s_2, \dots, s_a$) have been obtained. Note that m_{a+1} is only related to the first a spiking signals:

$$k_{a+1} = \sum_{i=1}^{a+1} p_{a+1-i}^{a+1} y_i, \quad (17)$$

$$m_{a+1} = U_{th} \sum_{i=1}^a q_{a-i}^{a+1} s_i + v_{th}, \quad (18)$$

where $p^{a+1} = (\tau^0, \tau^1, \dots, \tau^a)$, $q^{a+1} = (0, \tau^0, \tau^1, \dots, \tau^{a-1})$. Therefore, in the b -th iteration of PMBC, at least one new spiking signal (s_{a+1}) can be definitely obtained by $s_{a+1} = H_s(k_{a+1} - m_{a+1})$, thus the maximum number of iterations of PMBC is no more than L . Q.E.D.

In fact, due to the parallel computing mechanism of PMBC, the actual number of iterations is much smaller than L . As shown in Figure 2(b) in the main text, after only 5 iterations, the explicit spiking state exceeds 99%, whereas processing a 1024-dimensional input sequence serially would require 1024 iterations, highlighting the efficiency of PMBC. Detailed experimental proofs and analyses are shown in Section III-B in the main text. Based on the proof process for Assertion III.1, we can conclude that the distribution of k_t is closely related to y_t , where y_t is the output obtained from the previous SSM block, calculated through layer normalization. Therefore, if Dyn-SSM initializes $v_{th} = 1$ during training, the first iteration of PMBC can effectively determine that most of the spiking signals are zero. This significantly reduces the number of required iterations and accelerates training efficiency.

C. The Proof of Assertion III.3

Similarly to the proof of Assertion III.1, we utilize the Mathematical Induction Theory [1] to proof Assertion III.3. As the presupposition, we set $s_0 = 0$, $u_0 = 0$, and $R_0 = 0$, which is reasonable since there's no signal at $t = 0$ with the time step of the spiking signal s , membrane voltage u and input current I signal all considered as 1 to L , that is s_t, u_t , and $I_t, t = 1, \dots, L$. On this basis, according to Eqs. (9-10) in the main text ($u_t = \tau u_{t-1} + I_t - R_t U_{th}$, $R_t = \tau_r R_{t-1} + s_{t-1}$, $s_t = H(u_t - v_{th})$), we can transform the problem into proving the following equation first.

$$u_t = k_t - m_t + v_{th}, s_t = H_s(k_t - m_t), \quad (19)$$

$$\text{where } k_t = \sum_{i=1}^t \tau^{t-i} I_i, \text{ and} \quad (20)$$

$$m_t = U_{th} \sum_{i=1}^{t-1} \tau^{t-1-i} R_{i+1} + v_{th}, \text{ and} \quad (21)$$

$$R_{t+1} = \tau_r R_t + s_t, \quad (22)$$

Firstly, consider $t = 1$, we can obtain:

$$u_1 = \tau u_0 - R_1 U_{th} + I_1 \quad (23)$$

$$\xrightarrow{s_1 = H_s(u_1 - v_{th})} k_1 = I_1, \quad m_1 = v_{th}, \quad (24)$$

which is congruent to Eq. (22) with $t = 1$. Then, to verify that Eqs. (19-22) hold for all $t \in [1, L]$, we assume that Eqs. (19-22) hold for $t = t_1 \in [1, L-1]$, that is:

$$u_{t_1} = k_{t_1} - m_{t_1} + v_{th}, s_{t_1} = H_s(k_{t_1} - m_{t_1}), \quad (25)$$

$$\text{where } k_{t_1} = \sum_{i=1}^{t_1} \tau^{t_1-i} I_i, \text{ and} \quad (26)$$

$$m_{t_1} = U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} R_{i+1} + v_{th}, \text{ and} \quad (27)$$

$$R_{t_1+1} = \tau_r R_{t_1} + s_{t_1}, \quad (28)$$

based on this we have:

$$u_{t_1+1} = \tau u_{t_1} - R_{t_1+1} U_{th} + I_{t_1+1}, \quad (29)$$

$$s_{t_1+1} = H_s(u_{t_1+1} - v_{th}). \quad (30)$$

According to substituting the u_{t_1} in Eq. (25) into Eq. (29), we have:

$$u_{t_1+1} = \tau(k_{t_1} - m_{t_1} + v_{th}) - R_{t_1+1} U_{th} + I_{t_1+1} \quad (31)$$

$$= \tau \left(\sum_{i=1}^{t_1} \tau^{t_1-i} I_i + v_{th} \right) \quad (32)$$

$$- \tau \left(U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} R_{i+1} + v_{th} \right) \quad (33)$$

$$- R_{t_1+1} U_{th} + I_{t_1+1} \quad (34)$$

$$= \tau \sum_{i=1}^{t_1} \tau^{t_1-i} I_i + I_{t_1+1} - R_{t_1+1} U_{th} \quad (35)$$

$$- U_{th} \sum_{i=1}^{t_1-1} \tau^{t_1-1-i} R_{i+1} \quad (36)$$

$$= \sum_{i=1}^{t_1+1} \tau^{t_1+1-i} I_i + v_{th} \quad (37)$$

$$- \left(U_{th} \sum_{i=1}^{t_1} \tau^{t_1-i} R_{i+1} + v_{th} \right) \quad (38)$$

$$= k_{t_1+1} - m_{t_1+1} + v_{th} \quad (39)$$

Therefore, Eqs. (19-22) hold for $t = t_1 + 1 \in [2, L]$. According to the Mathematical Induction Theory [1], we can conclude that Eqs. (19-22) hold for all the $t \in [1, L]$. Subsequently, the parallel calculation form of the membrane potential of the LIF neuron with soft reset and refractory period can be expressed

TABLE I

THE HYPER-PARAMETERS OF OUR EXPERIMENTS ON THESE DATASETS. **H** DENOTES THE MODEL DIMENSION, **N** DENOTES THE STATE DIMENSION, **LR** DENOTES LEARNING RATE, **WD** DENOTES WEIGHT DECAY AND **BS** DENOTES THE BATCH SIZE. **BN** AND **LN** REFER TO BATCH NORMALIZATION AND LAYER NORMALIZATION.

Dataset	Depth	H	N	Norm	pNorm	Dropout	LR	BS	Epochs	WD	$(\Delta_{min}, \Delta_{max})$
sMNIST	2	128	64	LN	False	0.1	0.01	64	25	0.01	(0.001, 0.1)
psMNIST	4	128	64	LN	False	0.1	0.01	64	60	0.01	(0.001, 0.1)
sCIFAR10	4	128	64	LN	False	0.1	0.01	64	100	0.01	(0.001, 0.1)
ListOps	8	128	64	BN	False	0	0.01	50	40	0.05	(0.001, 0.1)
Text	6	256	64	BN	True	0	0.01	16	32	0.01	(0.001, 0.1)
Retrieval	6	256	64	BN	True	0	0.01	64	20	0.01	(0.001, 0.1)
Image	6	512	64	LN	False	0.1	0.01	50	200	0.01	(0.001, 0.1)
Pathfinder	6	256	64	BN	True	0	0.004	64	200	0.01	(0.001, 0.1)
Path-X	6	256	64	BN	True	0	0.0005	32	50	0.01	(0.0001, 0.01)
WT-103	16	1024	64	LN	True	0.1	0.0005	1	200	0.01	(0.001, 0.1)

as:

$$u_t = \sum_{i=1}^t \tau^{t-i} I_i - U_{th} \sum_{i=1}^t \tau^{t-i} R_i \quad (40)$$

$$= \sum_{i=1}^t \tau^{t-i} I_i - U_{th} \sum_{i=1}^t \tau^{t-i} \left(\sum_{j=1}^{i-1} \tau_r^{i-1-j} s_j \right) \quad (41)$$

$$= \sum_{i=1}^t \tau^{t-i} I_i - U_{th} \sum_{i=1}^{t-1} \sum_{j=0}^{t-1-i} \tau^j \tau_r^{t-1-i-j} s_i \quad (42)$$

$$= \sum_{i=1}^t \tau^{t-i} I_i - U_{th} \sum_{i=1}^{t-1} \sum_{j=0}^{t-1-i} (\tau/\tau_r)^j \tau_r^{t-1-i} s_i \quad (43)$$

$$s_t = H_s(u_t - v_{th}), \quad k_t = \sum_{i=1}^t \tau^{t-i} I_i \quad (44)$$

$$m_t = U_{th} \sum_{i=1}^{t-1} \sum_{j=0}^{t-1-i} (\tau/\tau_r)^j \tau_r^{t-1-i} s_i + v_{th} \quad (45)$$

Therefore, Assertion III.3 holds.

Q.E.D.

Apparently, Eqs. (11-13) in the main text degrade to the version of Dyn-SSM-RS without the refractory period when $\tau_r = 0$.

II. MORE DETAILS RELATED TO DYN-SSM

A. Detailed Description of Experimental Setup for Figure 2

For Figure 2(a) in the main text, both the soft reset mechanism and refractory period are considered in the LIF neuron of Dyn-SSM block trained with PMBC. The parameters τ , τ_r , U_{th} , and v_{th} are fixed as 0.3, 0.4, 1, and 0.5 respectively. The number of iterations in PMBC is set to 10 during training. The curve in Figure 2(a) in the main text shows the results with different iteration numbers in PMBC during inference.

For Figure 2(b) in the main text, only the soft reset mechanism is considered in the LIF neuron of the Dyn-SSM block trained with PMBC. The parameters τ , U_{th} , and v_{th} are fixed as 0.1, 1, and 3 respectively. The number of iterations in PMBC is set to 10 during training. After all the iterations of PMBC, the still fuzzy spiking signals are set to 1 by default, i.e., the spiking signals for these time steps are set to fire by default. The curve in Figure 2(b) in the main text shows

the explicit spiking state with different iteration numbers in PMBC during inference.

B. Comparison of Dyn-SSM and SpikingSSM [2]

According to Table I in the main text, SpikingSSM [2] has partial trainable dynamics since hard reset is rough and simplified with limited dynamic variables. In contrast, our method can train the neuron hyperparameters and temporal dynamics efficiently and explicitly in a parallel manner with the proposed PMBC, enabling a functional study of its impact on the network. In fact, the SDN in SpikingSSM breaks down the original hard reset mechanism into two steps: (1) constructing an additional dataset for offline training and fitting of the SDN, and (2) using the SDN for predictions during the training process on the target task or dataset.

As a result, the trainable dynamics in SpikingSSM [2] are somewhat constrained, particularly with respect to the dynamic parameter τ , leading to the following three limitations:

- The SDN requires external proxy training, which involves time-consuming preparation of sequence datasets and training in the initial stage.
- The SDN is designed to fit a fixed external network for a single and fixed value of τ , which may result in poor generalizability across different τ values. Consequently, the trainable τ values are limited to pre-prepared discrete ones. In theory, the SDN requires training a separate τ for every desired τ value, resulting in a very narrow range of truly "learnable" parameters, as they are all limited to discrete values.
- The SDN is not end-to-end during use, with operations like invocation and lookup restricting its flexibility, making it extremely challenging to extend to neuron-wise scenarios.

In comparison, Dyn-SSM offers the following advantages regarding dynamical properties and training cost:

- Dyn-SSM eliminates the need for additional datasets and offline training, enabling an end-to-end training process that reduces both time and cost.
- The parameter τ is continuously adjustable and trainable, enabling truly biologically inspired and plausible dynamical properties, which enhance the model's robustness and generalizability.

TABLE II
COMPARISON OF DYNAMICAL PROPERTIES ON PARAMETER τ BETWEEN SPIKINGSSMS AND DYN-SSM.

Methods	SpikingSSM	Dyn-SSM-1	Dyn-SSM-2
Accuracy (%) $\uparrow$	81.52	85.33	85.13
Spiking Rate (%) $\downarrow$	18.30	14.57	11.34

TABLE III
COMPARISON OF TRAINING TIME OF DIFFERENT METHODS. TRAINING WITH THE PMBC STRATEGY ACHIEVES SIGNIFICANT ACCELERATION, THE SPEED-UP RATIO AMPLIFIES WITH INCREASING SEQUENCE LENGTH.

Method	time of 100 iterations (s) $\uparrow$			
	$L = 1K$	$L = 2K$	$L = 4K$	$L = 8K$
Training with BPTT mozer2013focused	166.7	344.8	909.1	3333.0
Training with SLTT meng2023towards	136.9	303.0	833.3	3333.0
Training with PMBC (ours)	5.8	9.9	18.9	38.0
Speed-up Ratio	25.6 $\times$	32.2 $\times$	47.9 $\times$	81.7 $\times$

Note: all GPU timing results are wall-clock runtime measurements on conventional NVIDIA GPU hardware, demonstrating the practical computational acceleration of our PMBC strategy.

To provide a more detailed explanation, we conducted the following experiments: we compare our method with the SDN in SpikingSSM [2]. To better reflect the role of neuronal dynamics, we set all neurons in Dyn-SSM to share the same trainable τ within the range (0, 1), excluding the refractory mechanism. The experiments were performed on the SCIFAR-10 dataset, with PMBC iterations set to 10. We recorded the results under two settings: (1) where U_{th} and v_{th} are fixed at 1 (Dyn-SSM-1), and (2) where U_{th} and v_{th} are neuron-wise and trainable within the range (0, 1) (Dyn-SSM-2). For SpikingSSM, we use SDN that is trained from scratch with the SpikingSSM end-to-end, and 'SDN' uses the fixed pre-trained SDN. The results are shown in Table II. From Table II we can observe that trainable τ , U_{th} and v_{th} promote a better balance between accuracy and sparsity.

In addition, the ablation studies about hyper-parameters τ and τ_r , fire modes of fuzzy spiking singals, and the number of iterations M in PMBC, as detailed in the Section IV-E, Section IV-F, and Section IV-G in the main text, respectively, can further underscore and demonstrate the superiority of our method over SpikingSSMs [2].

It is worth mentioning that, compared to the original SSM and SpikingSSM [2], Dyn-SSM can be seen as an improvement based on biologically interpretable activation functions. Specifically, the introduction of the PMBC algorithm and interpretable dynamic neurons enables the spike-based activation in the network training process. This improvement in the activation function is highly meaningful.

Specifically, the activation function based on dynamic neurons can achieve better sparsity, which brings several advantages, such as: (i) It isolates the most crucial parts of the features, resulting in a sparser representation, which makes the model more robust. (ii) The variable-scale feature representation captures richer information. (iii) Sparse feature representations are more easily linearly separable.

In fact, this may be similar to the theoretical explanation of ReLU activation function, where fewer neurons are activated.

TABLE IV
THE COMPARISON OF PARALLELIZATION PERFORMANCE WITHOUT PMBC OR RESET MECHANISM.

Model	Soft Reset + PMBC	w/o PMBC	w/o Soft Reset
Accuracy (%) $\uparrow$	85.55	85.31	84.88
Spiking Rate (%) $\downarrow$	11.94	12.27	12.05

For theoretical benefits of feature sparsity induced by the ReLU function, please refer to the papers [3] and [4]. We will explore this idea more deeply in the future.

C. Ablation Study of PMBC and Reset mechanism

To show the rational design of our default Dyn-SSM (using the PMBC algorithm for the approximation of the original sequential computing caused by the Soft Reset mechanism), we did an ablation study on the reset mechanism and the PMBC algorithm, respectively. The model without PMBC (i.e., sequential computing) or without the Soft Reset mechanism both have a worse performance than the default model, both on expressivity (Accuracy) and efficiency (Spiking Rate). It shows that the reset mechanism is crucial for more abundant neural dynamics, while our PMBC algorithm can reach a significant acceleration without loss of performance.

III. MORE DETAILS RELATED TO EXPERIMENTS

A. Details of Datasets

1) *Sequential Vision Datasets*: The MNIST dataset [5] is a classic benchmark in machine learning, featuring 70,000 grayscale images of handwritten digits (0-9), with 60,000 samples for training and 10,000 for testing, each image sized at 28×28 pixels. It has long been a cornerstone for evaluating image classification models due to its simplicity and widespread availability. The sequential MNIST (sMNIST) dataset [6] transforms these 2D images into sequences of 784 elements by flattening the pixel grid into a 1D sequence. This transformation poses a more complex challenge, as models must process and retain information over longer time steps to accurately classify the digit. To further increase the difficulty, the permuted sequential MNIST (psMNIST) [6] applies a fixed random permutation to the pixel sequences, effectively scrambling their spatial order and disrupting any inherent structure. This variant demands even greater computational ability from models, as they must learn to extract meaningful features from a sequence with no obvious temporal or spatial coherence, making psMNIST a far more challenging task compared to the original sMNIST. The sCIFAR10 dataset is a sequential version of the CIFAR-10 dataset [7], where the original static images are processed in a temporal manner, typically by feeding the image pixels row-by-row or in a pre-defined sequence.

2) *LRA Benchmark*: The LRA benchmark [8] was specifically designed to evaluate the performance of sequence models in long-context scenarios, where capturing dependencies across extended sequences is crucial. It consists of six diverse tasks, with sequence lengths ranging from $1K$ to $16K$ steps,

covering multiple modalities including visual data, mathematical expressions, and natural language. These tasks are carefully curated to challenge models on various aspects of long-context comprehensions, such as text classification, document retrieval, image recognition, the pathfinder problem, and list operations (ListOps), making it a comprehensive testbed for assessing a model’s ability to process and reason over extended input sequences.

3) *WikiText-103*: The WikiText-103 dataset [9] is a large-scale corpus containing over 100 million tokens extracted from Wikipedia articles that have been rated as Good or Featured. Spanning a broad spectrum of topics and domains, it offers a rich variety of linguistic patterns and structures. Unlike many other datasets, WikiText-103 consists of full-length articles rather than isolated snippets, making it particularly well-suited for models designed to capture long-term dependencies across extended contexts. Due to its depth and diversity, it has become a pivotal benchmark for word-level language modeling, providing a robust testing ground for evaluating models’ capacity to understand and generate coherent text over lengthy sequences.

B. Details of Experimental Settings

The hyper-parameters τ and τ_r are set to 0.1 and 0.9, respectively. To ensure the threshold and refractory magnitude are positive during training, the trainable parameters v_{th} and U_{th} are computed by $\exp(v_{th})$ and $\exp(U_{th})$ with zero initialization (i.e., $\exp(v_{th})$ and $\exp(U_{th})$ are initialized as 1). Other parameters of Dyn-SSM blocks are initialized same as S4D-Lin [10]. Dyn-SSM is trained with Pytorch library on four NVIDIA A100-SXM4-80GB GPUs and AMD EPYC 7642 48-core CPUs, using AdamW optimization [11]. For sCIFAR10, sMNIST, psMNIST and LRA benchmarks, the model is trained by the cross-entropy loss [12] with accuracy (Acc) results reported, while the Perplexity results are reported for WikiText-103. The division of training and test data is consistent with [2].

Table I describes the particular training details of experiments on different tasks, where LRA benchmarks consist of six tasks, including ListOps, Text, Retrieval, Image, Pathfinder, and Path-X. **Depth** denotes the number of Dyn-SSM blocks, $pNorm$ denotes the pre-norm, and $WT - 103$ means WikiText-103 dataset.

C. Superiority of PMBC on Training Speed

We compare the training speed of Dyn-SSM, enhanced by our PMBC strategy, against traditional methods based on iterative LIF neurons, including Back-Propagation Through Time (BPTT) [13] and Spatial Learning Through Time (SLTT) [14], which use an optimized computational graph. The input consists of randomly generated 1-D sequences with various lengths of $L = 1K, 2K, 4K$, and $8K$, and a batch size of 64. All time measurements were conducted on a single NVIDIA A100-SXM4-80GB GPU. Through Table III, the speed-up ratio using PMBC increases with sequence length, achieving a nearly two-order acceleration at $8K$ when compared with. In

addition, we compare the training speed of PMBC with SpikingSSMs [2], which proposes a surrogate dynamic network (SDN) to approximate the dynamics of LIF neurons via a well-designed model. According to Table III, we can observe that our proposed PMBC method significantly outperforms SND, demonstrating the effectiveness of our approach.

D. Discussion Regarding the GPU Memory Usage

In this subsection, we conduct detailed peak memory consumption tests for our proposed PMBC, BPTT [13], and SLTT [14] across different sequence lengths L and PMBC iteration counts K , both at the single neuron level and for the full network. The results are summarized below.

To test the memory usage of a single neuron, we conducted the following simulation experiment: we randomly generated a tensor of size L , passed it through the neuron, and measured the resulting memory usage. The results are shown in Table V.

TABLE V
PEAK GPU MEMORY CONSUMPTION (MB) OF A SINGLE NEURON UNDER DIFFERENT SEQUENCE LENGTHS.

Method / Configuration	$L = 1K$	$L = 2K$	$L = 4K$	$L = 8K$
BPTT [13] (Serial)	20.2	39.7	79.5	161.2
SLTT [14] (Serial)	19.8	40.3	80.6	158.8
PMBC ($K=2$ iterations)	91.7	184.6	368.3	735.2
PMBC ($K=4$ iterations)	92.5	183.4	368.9	736.8
PMBC ($K=8$ iterations)	92.7	185.0	369.1	737.0

We have the key analysis and observations from the above results:

- PMBC introduces a predictable increase in peak memory compared to serial methods (approximately 4 $\times$), which is the expected trade-off for achieving up to 81.7 $\times$ training speedups (as shown in Table 3 in Supplementary Materials). This “space-for-time” trade-off remains highly favorable.
- Memory consumption scales linearly with sequence length L but is independent of the number of PMBC iterations K . This is because iterations are computed sequentially, so their intermediate states do not coexist in memory.
- BPTT [13] and SLTT [14] exhibit identical memory footprints, as they share the same forward computation graph; the only difference lies in gradient flow behavior during backpropagation.

Crucially, this overhead at the neuron level is negligible in the full network context, where the dominant memory consumption comes from the token mixer/projection layers, not the spiking neuron module itself.

Furthermore, we evaluated the total memory footprint of the full model. In this test, we configured BatchSize = 64 and $d_{model} = 128$. The results are presented in Table VI.

From the above results, we have the following observations:

- At the full network level, the relative overhead of PMBC becomes much smaller: the peak memory increase is only about 30–40% compared to serial training. This confirms that the additional memory required by PMBC is minor

TABLE VI
PEAK GPU MEMORY CONSUMPTION (MB) OF THE FULL NETWORK
UNDER DIFFERENT SEQUENCE LENGTHS (BATCHSIZE=64, $d_{\text{MODEL}}=128$).
THE MEMORY OVERHEAD INTRODUCED BY PMBC IS NEGLIGIBLE
COMPARED TO THE OVERALL NETWORK FOOTPRINT.

Network Configuration	$L = 1K$	$L = 2K$	$L = 4K$	$L = 8K$
Network + BPTT [13] (Serial)	1249.2	2471.9	4926.3	9832.9
Network + SLTT [14] (Serial)	1247.9	2472.6	4927.1	9834.1
Network + PMBC ($K=2$)	1646.3	3352.1	6786.8	13553.0
Network + PMBC ($K=4$)	1644.9	3355.7	6787.2	13554.4
Network + PMBC ($K=8$)	1648.6	3354.3	6785.0	13553.6

compared to the dominant costs of model parameters and mixer/projection activations.

- The scaling behavior remains predictable and linear with L , with no unexpected growth as the sequence length increases.

IV. DISCUSSION REGARDING THE ENERGY SAVING OF OUR PROPOSED ALGORITHM ON THE NEUROMORPHIC HARDWARE

From the perspective of computational paradigm compatibility, PMBC’s deterministic parallel computing model aligns closely with the architectural features of modern neuromorphic processors. Its fixed iteration count and predictable computational flow enable the algorithm to fully leverage programmable parallel computing cores in processors such as Intel Loihi 2 [15]. In particular, PMBC’s low precision requirements—often requiring only 8-bit or lower precision—are entirely consistent with the design philosophy of low-precision computational units commonly used in neuromorphic hardware [16].

In terms of hardware implementation feasibility, PMBC’s core computational modules can be effectively realized using existing neuromorphic hardware architectures. Parallel comparison operations can be performed using digital comparator arrays, the simple arithmetic operations required for boundary updates can be implemented via hardware pipelines [16], and the refractory accumulation process is compatible with existing leaky integrator circuits [17]. This implementation approach not only avoids the memory bottleneck issues inherent in traditional von Neumann architectures but also leverages optimized on-chip network communication mechanisms in neuromorphic processors [18].

Energy efficiency is a key advantage of neuromorphic hardware, and PMBC performs particularly well in this regard. The sparse spiking characteristics generated by the algorithm—with an average spiking rate typically below 10%—are perfectly compatible with hardware power gating techniques, enabling dynamic power management [19]. Additionally, the algorithm’s data locality features reduce long-distance data communication requirements, further lowering system energy consumption [15].

For real-time inference applications, PMBC demonstrates significant advantages. Its deterministic computational latency ensures predictable system response times, which is critical for real-time control systems. Preliminary simulation analysis suggests that on an FPGA platform, PMBC’s parallel implementation can achieve significant acceleration compared to

traditional serial methods. More importantly, the algorithm’s parallel nature ensures good scalability, allowing it to adapt to larger-scale neuromorphic systems in the future [17].

Of course, achieving full hardware deployment still requires addressing certain engineering challenges, such as adapting programming models, optimizing mixed-precision support, and refining dynamic power management. However, these challenges remain within the solvable scope of existing technological frameworks.

In summary, the deep alignment between PMBC and neuromorphic hardware in terms of computational paradigms, precision requirements, and energy efficiency characteristics provides a solid foundation for its deployment in practical systems. We believe that with further hardware optimization and system integration, PMBC holds promise as an efficient real-time computing solution for next-generation edge intelligent devices.

V. QUANTIZATION CONSISTENCY AND NEUROMORPHIC DEPLOYMENT FEASIBILITY

In this part, we have conducted dedicated experiments to verify whether the equivalence between parallel-trained PMBC and recurrent (serial) inference still holds after model quantization, especially focusing on whether firing/non-firing states remain consistent and whether performance degradation occurs.

To be specific, we evaluate the quantization robustness and parallel-recurrent consistency of PMBC on three standard sequential learning benchmarks—sMNIST, psMNIST, and sCIFAR10—chosen for their simplicity and interpretability in analyzing spiking dynamics, with a fixed batch size of 64 and sequence lengths of 784 for sMNIST/psMNIST and 1024 for sCIFAR10. We conduct experiments across three precision settings: full-precision, 8-bit, and 4-bit quantization, using the standard PTQ pipeline in [20] without additional quantization-aware training. We measure two metrics to assess performance and consistency: overall classification accuracy to evaluate end-task performance, the Spike Consistency Rate (SCR) between PMBC parallel training and recurrent inference to directly quantify the equivalence of spike patterns under the two modes, which is defined as:

$$\text{SCR} = \frac{1}{T} \sum_{t=1}^T \mathbb{I}[s_t^{\text{parallel}} = s_t^{\text{recurrent}}]. \quad (46)$$

The results are shown as follows:

From Table VII, our key findings are as follows:

(1) Firing and non-firing state consistency is well preserved after quantization

We quantized all trainable parameters (including thresholds V_{th} , U_{th} , time constants τ , τ_r) to 8-bit and 4-bit integers, which are typical settings for neuromorphic chips. We then compared the firing sequences predicted by parallel PMBC training and those generated by real recurrent inference.

The spike consistency rate remains about 97% even under 4-bit quantization, confirming that the boundary compression mechanism is highly robust to numerical precision changes.

TABLE VII
QUANTIZATION PERFORMANCE AND PARALLEL-RECURRENT
CONSISTENCY ACROSS TASKS. EVEN UNDER 4-BIT QUANTIZATION, THE
SPIKE CONSISTENCY RATE (SCR) REMAINS HIGH, CONFIRMING THAT
PMBC MAINTAINS ITS EQUIVALENCE PROPERTY.

Model & Quantization	Task	Accuracy (%)	SCR (%)
Full Precision	sMNIST	99.53	99.81
8-bit Quantization	sMNIST	99.10	98.92
4-bit Quantization	sMNIST	98.37	97.86
Full Precision	psMNIST	97.89	99.76
8-bit Quantization	psMNIST	97.31	98.75
4-bit Quantization	psMNIST	96.44	97.13
Full Precision	sCIFAR10	85.67	99.68
8-bit Quantization	sCIFAR10	85.12	98.61
4-bit Quantization	sCIFAR10	84.39	96.94

(2) No obvious dynamics divergence or performance collapse

We evaluated classification accuracy on sMNIST, psMNIST, and sCIFAR10 under quantized settings. The performance drop after 8-bit quantization is less than 1%, and under 4-bit quantization it remains within 2%. This demonstrates that the small inconsistencies in spike states do not lead to catastrophic dynamics divergence.

(3) Theoretical justification

PMBC’s boundary compression relies on monotonic, order-based comparisons rather than high-precision arithmetic. Since quantization only scales values but largely preserves their relative order, the core decision logic of “ $k_t > m_t^{\text{low}} + v_{\text{th}}$ ” remains nearly unchanged. Thus, the equivalence between parallel training and recurrent inference is naturally quantization-robust.

In summary, based on these results, we can safely conclude that PMBC maintains strong consistency between parallel training and recurrent inference even after quantization, and can be reliably deployed on neuromorphic hardware without significant performance degradation.

REFERENCES

- [1] J. A. Bather, “Mathematical induction,” 1994.
- [2] S. Shen, C. Wang, R. Huang, Y. Zhong, Q. Guo, Z. Lu, J. Zhang, and L. Leng, “Spikingssms: Learning long sequences with sparse and parallel spiking state space models,” *arXiv preprint arXiv:2408.14909*, 2024.
- [3] V. Nair and G. E. Hinton, “Rectified linear units improve restricted boltzmann machines,” in *Proceedings of the 27th international conference on machine learning (ICML-10)*, 2010, pp. 807–814.
- [4] A. L. Maas, A. Y. Hannun, A. Y. Ng *et al.*, “Rectifier nonlinearities improve neural network acoustic models,” in *Proc. icml*, vol. 30, no. 1. Atlanta, GA, 2013, p. 3.
- [5] L. Deng, “The mnist database of handwritten digit images for machine learning research [best of the web],” *IEEE Signal Processing Magazine*, vol. 29, no. 6, pp. 141–142, 2012.
- [6] Q. V. Le, N. Jaitly, and G. E. Hinton, “A simple way to initialize recurrent networks of rectified linear units,” *arXiv preprint arXiv:1504.00941*, 2015.
- [7] A. Krizhevsky, G. Hinton *et al.*, “Learning multiple layers of features from tiny images,” 2009.
- [8] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, and D. Metzler, “Long range arena: A benchmark for efficient transformers,” *arXiv preprint arXiv:2011.04006*, 2020.
- [9] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” *arXiv preprint arXiv:1609.07843*, 2016.

- [10] A. Gu, K. Goel, A. Gupta, and C. Ré, “On the parameterization and initialization of diagonal state space models,” *Advances in Neural Information Processing Systems*, vol. 35, pp. 35 971–35 983, 2022.
- [11] I. Loshchilov, “Decoupled weight decay regularization,” *arXiv preprint arXiv:1711.05101*, 2017.
- [12] A. Mao, M. Mohri, and Y. Zhong, “Cross-entropy loss functions: Theoretical analysis and applications,” in *International conference on Machine learning*. PMLR, 2023, pp. 23 803–23 828.
- [13] M. C. Mozer, “A focused backpropagation algorithm for temporal pattern recognition,” in *Backpropagation*. Psychology Press, 2013, pp. 137–169.
- [14] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z.-Q. Luo, “Towards memory- and time-efficient backpropagation for training spiking neural networks,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 6166–6176.
- [15] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain *et al.*, “Loihi: A neuromorphic manycore processor with on-chip learning,” *Ieee Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [16] C. Frenkel, M. Lefebvre, J.-D. Legat, and D. Bol, “A 0.086-mm² 12.7-pj/sop 64k-synapse 256-neuron online-learning digital spiking neuromorphic processor in 28-nm cmos,” *IEEE transactions on biomedical circuits and systems*, vol. 13, no. 1, pp. 145–158, 2018.
- [17] S. B. Furber, F. Galluppi, S. Temple, and L. A. Plana, “The spinnaker project,” *Proceedings of the IEEE*, vol. 102, no. 5, pp. 652–665, 2014.
- [18] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan, B. L. Jackson, N. Imam, C. Guo, Y. Nakamura *et al.*, “A million spiking-neuron integrated circuit with a scalable communication network and interface,” *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [19] G. Indiveri, B. Linares-Barranco, T. J. Hamilton, A. v. Schaik, R. Etienne-Cummings, T. Delbruck, S.-C. Liu, P. Dudek, P. Häflicher, S. Renaud *et al.*, “Neuromorphic silicon neuron circuits,” *Frontiers in neuroscience*, vol. 5, p. 73, 2011.
- [20] M. Nagel, M. Fourmarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort, “A white paper on neural network quantization,” *arXiv preprint arXiv:2106.08295*, 2021.